USER AND TECHNICAL DOCUMENTATION

ShopOpt

Online Shopping Product Optimizer

A budget-aware shopping assistant that applies the 0/1 Knapsack algorithm, sorting algorithms and binary search to a catalogue of 32 branded products across six sections.

Item	Detail
Product	ShopOpt, single-page web application (one HTML file)
Version	1.0
Document date	3 October 2026
Audience	Students, evaluators, developers and end users
Technology	HTML, CSS and vanilla JavaScript; optional REST backend

Contents

1. Introduction....3
2. Problem statement and objectives....4
3. Scope and feature summary....5
4. System architecture....6
5. Data model and application state....7
6. Product catalogue....8
7. Value model....9
8. 0/1 Knapsack: theory....10
9. 0/1 Knapsack: implementation and worked example....11
10. Optimizer module and results....12
11. Live visualisation....13
12. Sorting Lab....14
13. Binary search....15
14. Analytics and complexity....16
15. Offers, sections, search and voice....17
16. Authentication and backend API....18
17. User guide....19
18. Interface, theming and accessibility....20
19. Testing and validation....21
20. Limitations, future work and conclusion....22

Conventions. Currency amounts are written as “Rs.” followed by the amount (the application itself displays the rupee symbol). Names such as *Optimizer* and *Sorting Lab* refer to the navigation pages inside the application. All figures quoted in this document were produced by running the application’s own algorithms on its built-in catalogue.

1. Introduction

ShopOpt is an interactive web application that helps a shopper decide **what to buy within a fixed budget**. Instead of leaving the decision to guesswork, it treats the shopper's basket as a classic optimisation problem and solves it exactly with dynamic programming. The shopper chooses a budget and a goal, such as best ratings, highest utility, largest discounts or a weighted combination, and the application returns the basket that delivers the greatest total value without exceeding the budget.

The application is also a teaching tool. Every algorithm it uses is written out explicitly, without library shortcuts, and can be watched step by step. The knapsack table fills one product at a time, the sorting bars move as elements are compared and swapped, and binary search narrows its range on screen. A separate analytics page measures how long each algorithm takes in the user's own browser.

1.1 Purpose of this document

This document describes what ShopOpt does, how it is built and how to use it. It covers the data model, the mathematics behind the value score, the algorithms and their complexity, the user interface, the optional backend contract, test cases, and the known limitations of the current version.

1.2 Key facts at a glance

Aspect	Value
Products in catalogue	32 across 6 sections
Application pages	8: Dashboard, Products, Special offers, Optimizer, Sorting Lab, Binary Search, Analytics, Complexity
Algorithms	0/1 Knapsack (dynamic programming), bubble, selection, insertion and merge sort, binary search
Interface languages	20 selectable, with search labels translated for 6
Delivery	One self-contained HTML file, runs in any modern browser

2. Problem statement and objectives

2.1 The shopper's problem

Online catalogues present hundreds of items, each with a price, a rating and a discount. A shopper with a fixed budget faces a combinatorial choice: with only 32 products there are more than four billion possible baskets (2 to the power 32), and the best one is rarely the obvious one. Picking the highest-rated items first often exhausts the budget on a few expensive products, while picking the cheapest items first may leave useful value on the table.

2.2 Formal statement

Given a set of n products, each with a price (its weight) and a value measured by the chosen goal, and a budget B, select a subset of products whose total price does not exceed B and whose total value is as large as possible. Each product can be chosen at most once. This is the **0/1 Knapsack problem**, which is known to be solvable in pseudo-polynomial time with dynamic programming.

2.3 Objectives

- **Optimise** a basket for four selectable goals: rating, utility, discount or a weighted combination of the three.
- **Respect the budget** exactly, reporting cost, remaining amount and percentage of budget used.
- **Explain** each decision by showing the include-versus-exclude comparison and the filled table.
- **Demonstrate** sorting and searching on the same catalogue, with comparison and swap counts.
- **Measure** real execution time and relate it to theoretical complexity.
- **Remain accessible** by supporting voice search, 20 languages, light and dark themes, keyboard focus and mobile screens.

> **Design principle**
> Correctness first. Every number shown to the user, including the maximum value, the savings and the operation counts, is computed by the same code that makes the decision, so the display can never disagree with the result.

3. Scope and feature summary

ShopOpt is a front-end application. All optimisation, sorting and searching run inside the browser; a server is optional and used only for accounts and persistence. The table below lists the features by page.

Page	What it provides
Dashboard	Four key figures (budget, products available, best value found, total savings), a budget gauge, special-offer strip, six section tiles and a short how-it-works card.
Products	Chip bar to filter by section, section banner with brand list, product cards with image, rating, utility, price, discount and value score. Add, delete, select and upload-photo actions.
Special offers	Six section offers with codes, a member bonus for logged-in users and a Hot deals grid of products with 25% discount or more.
Optimizer	Goal selector, adjustable combined weights, budget input, the Optimize button, live knapsack table, final basket and explanation.
Sorting Lab	Choose key (price, rating, utility, discount), order and algorithm; animated bars with comparison, swap and time counters.
Binary Search	Search the sorted price list for a target; each step shows the active range and the midpoint.
Analytics	Measured execution time, input size and operation count for knapsack, sorting and binary search, with a Re-measure button.
Complexity	Cards giving time and space complexity of every algorithm with a plain-language explanation.

3.1 Out of scope

Payment, real inventory, delivery, and a production coupon engine are not part of this prototype. Offer codes are displayed but not applied to prices, and the medicines section carries a notice that it is a demonstration catalogue only.

4. System architecture

The application is a single-page design in one HTML file. A small global object holds the entire application state, a catalogue array holds the products, and each page is a function that returns an HTML string. Whenever state changes, a single render function rebuilds the visible page and restores the scroll position.

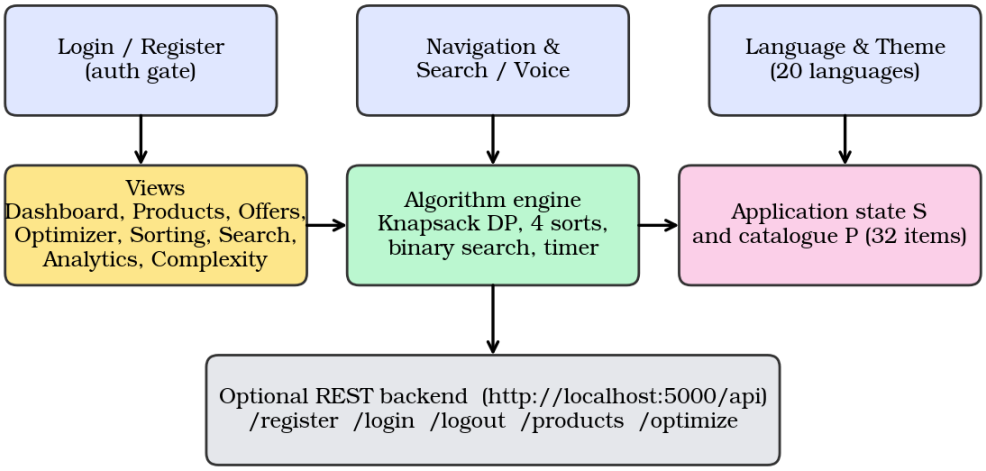


Figure 1. High-level structure of ShopOpt

4.1 Layers

- **Presentation layer.** A fixed sidebar for navigation, a header with title, search box, microphone button, language selector, theme toggle and login button, and a main area where the current view is drawn.
- **State layer.** One object, S, stores language, category filter, search text, signed-in user, current page, goal, budget, weights, selected product ids, the latest result, animation step and timer, measured performance and error text.
- **Algorithm layer.** Pure functions: knapsack, sortRun, bsearch and a timing helper. They take data in and return results, with no access to the page.
- **Service layer.** One asynchronous helper, api, wraps fetch, attaches a Bearer token when signed in and converts failures into readable messages.

The design keeps the algorithms independent of the interface, which makes them easy to test and to reuse.

5. Data model and application state

5.1 Product record

Every product in the catalogue is a record with the same nine fields. Fields are validated when a product is added through the interface.

Field	Type	Meaning and valid range
id	number	Unique identifier. Seed products use 1 to 32; products added by the user use a timestamp, or the server-assigned id when connected.
name	text	Display name, for example “G102 Gaming Mouse”.
cat	text	One of the six sections.
brand	text	Brand label shown on the product image, “Custom” for user-added items.
price	number	Positive multiple of 100 (in rupees). This is the knapsack weight.
rating	number	0 to 5.
utility	number	0 to 100. A subjective usefulness score.
discount	number	0 to 90 (percent) when adding; seed values range from 5 to 40.
icon	text	Key selecting the built-in vector illustration; a custom icon is used for new products.

5.2 Application state

State key	Purpose
page, cat, q	Current page, section filter and search text.
obj, budget, w	Chosen goal, budget (default Rs. 10,000) and combined weights (default 40, 30, 30).
sel, res	Set of selected product ids and the latest knapsack result including its table.
step, timer, speed	Animation position, interval handle and delay (600, 250 or 80 ms).
perf	Latest measured timings for knapsack, sort and binary search.
user, token, server	Signed-in user, session token and whether the server is in use.

Selecting a product on the Products page toggles its id in the selection set, and the budget gauge shows the total price of the current selection. After optimisation, the selection is replaced by the optimal basket.

6. Product catalogue

The built-in catalogue contains 32 products: **11 Electronics, 5 Clothing, and 4 each of Beauty, Groceries, Medicines and Pets**. The value score in the last column is the fixed 40/30/30 blend described in Section 7.

#	Product	Section	Brand	Price	Rtg	Util	Disc.
1	Watch SE	Electronics	Apple	Rs. 4,000	4.4	95	10%
2	WH-CH520 Headphones	Electronics	Sony	Rs. 2,500	4.6	90	20%
3	K2 Mechanical Keyboard	Electronics	Keychron	Rs. 3,000	4.7	85	15%
4	G102 Gaming Mouse	Electronics	Logitech	Rs. 1,500	4.5	75	25%
5	Flip 6 Speaker	Electronics	JBL	Rs. 2,000	4.3	70	30%
6	C270 HD Webcam	Electronics	Logitech	Rs. 1,500	4.1	60	12%
7	20000mAh Power Bank	Electronics	Xiaomi	Rs. 1,000	4.5	55	18%
8	6-in-1 USB-C Hub	Electronics	UGREEN	Rs. 1,000	4.2	50	22%
9	Aluminium Laptop Stand	Electronics	Portronics	Rs. 1,500	4.6	65	8%
10	LED Desk Lamp	Electronics	Philips	Rs. 1,000	4.0	40	35%
11	Smart Band 8	Electronics	Xiaomi	Rs. 2,500	4.2	72	28%
12	Laptop Backpack	Clothing	Wildcraft	Rs. 2,000	4.5	68	16%
13	Air Max Sneakers	Clothing	Nike	Rs. 4,000	4.5	88	30%
14	511 Slim Jeans	Clothing	Levi's	Rs. 2,500	4.3	70	25%
15	Cotton T-Shirt	Clothing	H&M	Rs. 800	4.2	55	40%
16	Puffer Jacket	Clothing	Zara	Rs. 3,500	4.4	82	35%
17	Matte Lipstick	Beauty	Lakmé	Rs. 600	4.4	60	30%
18	Hydrating Moisturiser	Beauty	Nivea	Rs. 500	4.5	58	20%
19	Eau de Parfum 100ml	Beauty	Park Avenue	Rs. 2,500	4.6	75	25%
20	Kajal Pencil	Beauty	Maybelline	Rs. 300	4.2	50	35%
21	Whole Wheat Atta 5kg	Groceries	Aashirvaad	Rs. 300	4.6	80	10%
22	Full Cream Milk 1L	Groceries	Amul	Rs. 100	4.7	70	5%
23	Sunflower Oil 1L	Groceries	Fortune	Rs. 200	4.3	62	12%
24	Basmati Rice 5kg	Groceries	India Gate	Rs. 600	4.5	78	18%
25	Paracetamol 650 Strip	Medicines	Dolo	Rs. 100	4.5	85	10%
26	Antiseptic Liquid 500ml	Medicines	Dettol	Rs. 300	4.6	70	15%
27	Pain Relief Spray	Medicines	Volini	Rs. 400	4.4	65	20%
28	Digital Thermometer	Medicines	Omron	Rs. 500	4.5	60	18%
29	Dry Dog Food 3kg	Pets	Pedigree	Rs. 1,500	4.6	85	15%
30	Cat Food Pouches	Pets	Whiskas	Rs. 500	4.5	70	10%
31	Chew Toy Bone	Pets	Kong	Rs. 300	4.2	50	25%
32	Grooming Brush	Pets	Trixie	Rs. 400	4.3	45	20%

Rtg = rating out of 5; Util = utility score out of 100; Disc. = discount percentage.

7. Value model

The optimiser needs a single number for each product, called its **value**, that expresses how much the shopper gains by including it. ShopOpt offers four definitions, selected by the goal buttons on the Optimizer page. To combine measures on different scales, each is first normalised to a 0 to 100 range.

7.1 Normalisation

Measure	Normalised score	Reasoning
Utility	u = utility	Already on a 0 to 100 scale.
Rating	r = rating ÷ 5 × 100	A 5-star product scores 100.
Discount	d = discount ÷ 50 × 100	A 50% discount scores 100; larger values exceed 100.

7.2 The four goals

Goal	Value of a product	Example: G102 Gaming Mouse
Rating	round(rating × 20)	round(4.5 × 20) = 90
Utility	utility	75
Discount	discount	25
Combined	round((wu·u + wr·r + wd·d) ÷ 100)	weights 40/30/30 → 72

With the default weights the combined value of the mouse is (0.40 × 75) + (0.30 × 90) + (0.30 × 50) = 30 + 27 + 15 = 72. The three weights always total 100: when the shopper moves one slider, the other two are rescaled in proportion so the total stays at 100%.

7.3 Value score on product cards

Each card shows a “Value score” that always uses the fixed 40/30/30 blend, independent of the selected goal. This gives a quick, comparable indicator while browsing. In the catalogue the highest scores are 80 for the Air Max Sneakers and the Puffer Jacket, followed by 76 for the WH-CH520 Headphones; the lowest are 56 for the C270 HD Webcam and the Grooming Brush.

> **Why integers?**
> Values are rounded to whole numbers so the dynamic-programming table stays compact and easy to read. The trade-off is that very similar products can tie, in which case the earlier product in the catalogue is preferred by the include test.

8. 0/1 Knapsack: theory

In the 0/1 Knapsack problem each item is either taken whole or left behind; it cannot be split and cannot be taken twice. A greedy rule, such as picking by value per rupee, can miss the optimum because it cannot undo an early choice. Dynamic programming avoids this by solving every smaller budget first and building up.

8.1 Recurrence

Let dp[i][c] be the best total value achievable using only the first i products with a budget of c units. For product i with weight w and value v:

```
dp[0][c] = 0                          for every c
dp[i][c] = dp[i-1][c]                  if w > c
dp[i][c] = max( dp[i-1][c],               (exclude product i)
              v + dp[i-1][c-w] )             (include product i)
```

Each cell asks one question: is it better to skip this product, keeping the best result from before, or to include it and add its value to the best result achievable with the money that remains? The answer for the whole problem sits in the last row at the full budget, dp[n][W].

8.2 Why it is correct

The problem has **optimal substructure**: an optimal basket for budget c that includes product i must contain an optimal basket for the remaining budget c minus w among the earlier products. It also has **overlapping sub-problems**, because the same (i, c) pair is needed by many larger states. Storing each result once turns an exponential search over 2 to the power n subsets into a table of n × W cells.

8.3 Complexity

Quantity	Bound	In ShopOpt (32 items, Rs. 10,000)
Time	O(n·W)	3,232 cell evaluations
Space	O(n·W)	3,333 stored cells (33 rows × 101 columns)
Brute force for comparison	$O(2^n)$	4,294,967,296 subsets

The bound is called pseudo-polynomial because W is a numeric value, not the length of the input. A budget of Rs. 10,00,000 would need a table 100 times wider, which is why ShopOpt scales prices down by 100 before building the table.

9. 0/1 Knapsack: implementation and worked example

9.1 How the application implements it

1. **Filter.** Products priced above the budget are removed, since they can never fit.
2. **Scale.** The unit is U = 100, so capacity W = floor(budget ÷ 100) and each weight is round(price ÷ 100). A Rs. 10,000 budget becomes W = 100.
3. **Fill.** For i = 1 to n and c = 0 to W the recurrence above is applied; a counter records every cell evaluated.
4. **Backtrack.** Starting at (n, W), if dp[i][c] differs from dp[i-1][c] then product i was taken, so it is recorded and c is reduced by its weight. Walking up to row 1 yields the basket.
5. **Report.** The maximum value, cost, remaining budget, savings and the table are handed to the display.

9.2 Worked example

To see the mechanism on a small scale, take four products and a budget of Rs. 800 (W = 8 units): Paracetamol Strip (weight 1, value 85); Chew Toy Bone (weight 3, value 50); Cat Food Pouches (weight 5, value 70); Grooming Brush (weight 4, value 45).

Product \ Budget	0	1	2	3	4	5	6	7	8
None	0	0	0	0	0	0	0	0	0
Paracetamol Strip	0	85	85	85	85	85	85	85	85
Chew Toy Bone	0	85	85	85	135	135	135	135	135
Cat Food Pouches	0	85	85	85	135	135	155	155	155
Grooming Brush	0	85	85	85	135	135	155	155	180

Table 1. DP table for the worked example (budget in units of Rs. 100)

The bottom-right cell holds the answer, **180**. Backtracking from that cell selects **Paracetamol Strip, Chew Toy Bone and Grooming Brush**, whose weights sum to 8 units and whose values sum to 180. A greedy “highest value first” rule would take the strip and the Cat Food Pouches for only 155, missing this better combination; the table proves that no other combination does better within 8 units.

10. Optimizer module and results

The Optimizer page ties the algorithm to the interface. The shopper picks a goal, enters a budget in steps of Rs. 100 and presses **Optimize my cart**. The application filters affordable products, runs the knapsack, measures its average running time over 20 repetitions, selects the winning products and starts the animation.

10.1 Results on the built-in catalogue

Goal	Budget	Items	Best value	Cost	Discount value captured
Utility	Rs. 10,000	18	1173	Rs. 9,900	Rs. 2,082
Rating	Rs. 10,000	18	1602	Rs. 9,900	Rs. 1,827
Discount	Rs. 10,000	18	378	Rs. 9,900	Rs. 2,377
Combined	Rs. 10,000	18	1158	Rs. 9,900	Rs. 2,252
Utility	Rs. 5,000	13	858	Rs. 4,700	Rs. 902
Combined	Rs. 20,000	24	1546	Rs. 19,900	Rs. 4,192
Combined	Rs. 3,000	10	630	Rs. 3,000	Rs. 562

Table 2. Optimal baskets produced by the knapsack for several goals and budgets

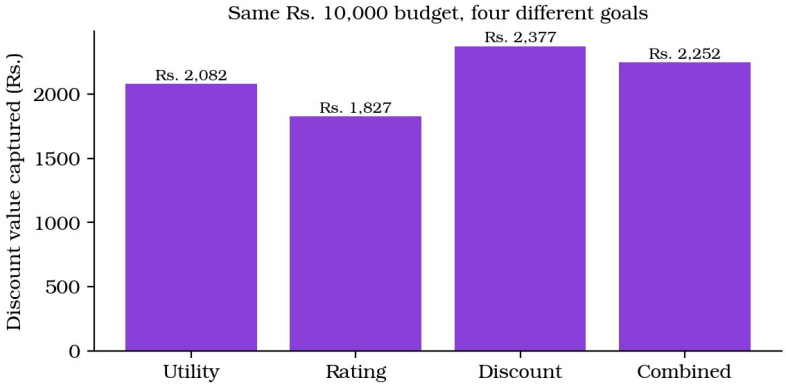


Figure 2. The discount-focused goal captures the most discount value for the same budget

The same budget gives different baskets for different goals. At Rs. 10,000, the utility goal includes the Gaming Mouse and Power Bank, the rating goal prefers the Aluminium Laptop Stand, and the discount goal brings in the USB-C Hub and LED Desk Lamp. In every run the total cost stays within the budget; at Rs. 10,000 the optimiser leaves only Rs. 100 unspent.

> **Reading the result**
>
> A larger “best value” is not comparable across goals, because each goal uses a different scale. Compare baskets for the same goal and budget only.

11. Live visualisation

After optimisation the page shows a **Live knapsack visualization** panel. It lets the shopper watch the dynamic-programming table grow one product at a time, so the algorithm can be understood rather than taken on trust.

11.1 Controls

Control	Behaviour
Play	Starts or resumes the automatic animation from the current step.
Pause	Stops the timer so the current table can be studied.
Next step	Pauses and advances exactly one product.
Reset	Pauses and returns to step 0, an empty table.
Speed	Slow (600 ms), Normal (250 ms) or Fast (80 ms) per step.

11.2 What is shown at each step

- A sentence naming the product being processed, its price and its value for the active goal.
- Two side-by-side boxes, **INCLUDE** and **EXCLUDE**. The include box shows the arithmetic (value plus the earlier best for the leftover budget); the exclude box shows the previous best. The winning box is highlighted green.
- A final decision line, either “INCLUDE” or “EXCLUDE”, and the message “does not fit” when the product is heavier than the whole budget.
- The table itself, with one row added per step. The current cell at full budget is coloured blue while running, and the final answer cell turns green when the animation finishes.

11.3 Completion summary

When the last product is processed, three panels appear. The **Optimization complete** card shows maximum value, total cost, remaining budget and number of products, with an OPTIMAL badge. The **Selected basket** table lists each chosen product with price, value, rating and discount, followed by bars showing each product’s contribution. Finally, **Why were these products selected?** states in plain language that the basket stays within budget, maximises the chosen goal, and was found by comparing all budget-and-product states.

For the default run (utility, Rs. 10,000) the panel reports a maximum value of 1173, a cost of Rs. 9,900, remaining Rs. 100, 18 products and 3,333 states examined.

12. Sorting Lab

The Sorting Lab sorts the 32 catalogue values for the chosen key (price, rating, utility or discount) in ascending or descending order. The shopper picks one of four algorithms and presses **Sort now**. Frames are recorded during a first run and replayed every 120 ms, with compared bars shown in orange and the finished array in green. A second, un-animated run repeated 200 times supplies the timing.

Algorithm	Idea	Time	Notes
Bubble	Swap adjacent out-of-order pairs repeatedly	O(n²)	Easy to follow, many swaps
Selection	Find the minimum of the rest and place it	O(n²)	Fewest swaps (at most n−1)
Insertion	Insert each element into the sorted prefix	O(n²)	Fast on nearly sorted data
Merge	Split, sort halves, merge them	O(n log n)	Needs extra O(n) space

12.1 Measured operation counts

The results below come from sorting the 32 prices in ascending order. Because the application counts comparisons and swaps, they are deterministic and independent of the computer used.

Algorithm	Comparisons	Swaps / writes	Total operations
Bubble	496	353	849
Selection	496	26	522
Insertion	376	353	729
Merge	117	160	277

Table 3. Operation counts for 32 prices

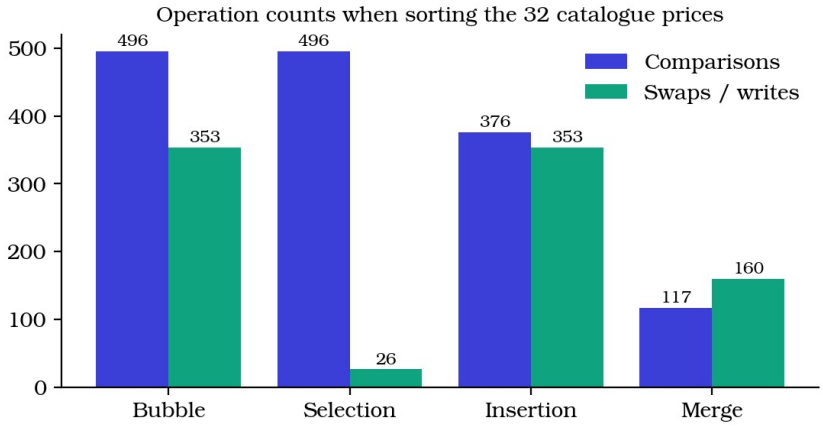


Figure 3. Merge sort needs about a quarter of the comparisons of the quadratic sorts

Bubble and selection sort both make 496 comparisons, which is n(n−1)/2 for n = 32, but selection performs only 26 swaps against 353 for bubble sort. Merge sort makes 117 comparisons, which is why it is also used to prepare the data for binary search.

13. Binary search

Binary search finds a target in a **sorted** list by repeatedly halving the range that could contain it. The page first sorts the 32 prices with merge sort, then searches for the price typed by the shopper (default Rs. 2,500).

13.1 Algorithm

```
lo = 0, hi = n - 1
while lo <= hi:
    mid = floor((lo + hi) / 2)
    if a[mid] == target:  return mid
    if a[mid] <  target:  lo = mid + 1      (search right half)
    else:                 hi = mid - 1      (search left half)
return not found
```

13.2 Sorted price list

0:100 1:100 2:200 3:300 4:300 5:300 6:300 7:400 8:400 9:500 10:500 11:500 12:600 13:600
14:800 15:1000 16:1000 17:1000 18:1500 19:1500 20:1500 21:1500 22:2000 23:2000 24:2500
25:2500 26:2500 27:2500 28:3000 29:3500 30:4000 31:4000

13.3 Trace: target Rs. 2,500 (found)

Step	Low	High	Middle (value)	Decision
1	0	31	15 (1000)	Search right half
2	16	31	23 (2000)	Search right half
3	24	31	27 (2500)	Match, target found at index 27

13.4 Trace: target Rs. 2,700 (not found)

Step	Low	High	Middle (value)	Decision
1	0	31	15 (1000)	Search right half
2	16	31	23 (2000)	Search right half
3	24	31	27 (2500)	Search right half
4	28	31	29 (3500)	Search left half
5	28	28	28 (3000)	Search left half

The page shows the active range in blue, the midpoint in yellow and the discarded elements faded. The first search needs only 3 comparisons and the unsuccessful one 5, compared with up to 32 for a straight scan. Since the list contains repeated prices, the index returned is whichever matching element the midpoint lands on first; the result is correct even though it may not be the lowest index.

14. Analytics and complexity

The **Analytics** page runs and times each algorithm in the user's browser using the high-resolution performance clock: knapsack averaged over 20 runs, merge sort over 200 runs and binary search over 2,000 runs. Each row shows the average time, input size, operations and a specific count (DP states, comparisons or search steps). Pressing **Re-measure** repeats the experiment. Absolute times depend on the device, so the operation counts are the dependable comparison.

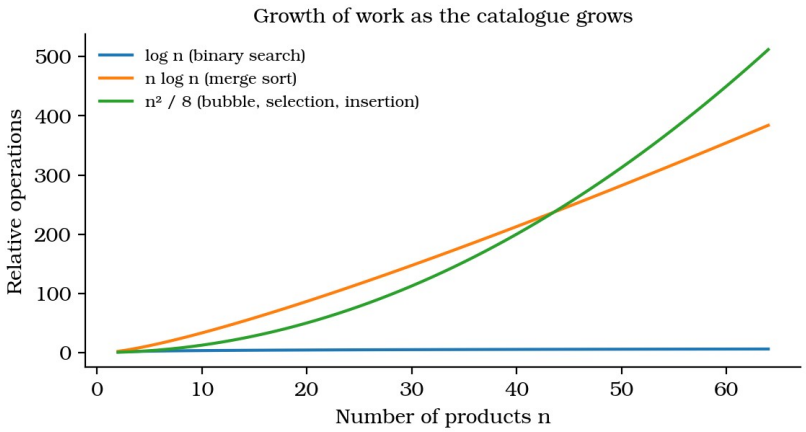


Figure 4. Why logarithmic and n log n algorithms scale better than quadratic ones

14.1 Complexity summary

Algorithm	Time	Space	Plain-language meaning
0/1 Knapsack	O(nW)	O(nW)	One cell per product and budget unit
Binary search	O(log n)	O(1)	1,000 items need about 10 steps
Merge sort	O(n log n)	O(n)	Split, sort, merge
Bubble / selection / insertion	O(n²)	O(1)	Simple but slow on large lists

The **Complexity** page presents the same information as four cards. A key lesson from the data is that doubling the catalogue roughly doubles the knapsack work (n doubles), but quadruples the work of the simple sorts, while binary search adds only a single extra step.

15. Offers, sections, search and voice

15.1 Sections

A chip bar and sidebar list the six sections, each with its own colour and emoji: Clothing, Electronics, Beauty, Groceries, Medicines and Pets. Choosing a section shows a coloured banner with the product count, brand list and a strip of illustrations. The Medicines banner adds a reminder that the catalogue is a demonstration and that a pharmacist should be consulted before taking any medicine.

15.2 Special offers

Section	Offer	Description	Code
Clothing	Flat 30% off sneakers and jackets	Seasonal sale	STYLE30
Electronics	Up to 25% off audio and wearables	Headphones, speakers, watches	TECH25
Beauty	Buy 2, get 15% off	All beauty products	GLOW15
Groceries	Rs. 100 off on Rs. 1,000+	Staples, dairy, oils	FRESH100
Medicines	Free delivery on health essentials	Orders above Rs. 300	CARE300
Pets	20% off first pet food order	Dog and cat food	PAWS20

Logged-in users also see a **Member bonus** card (code MEMBER5, extra 5% off); signed-out users see an invitation to log in. The Special offers page adds a **Hot deals** grid listing the 12 products that carry a discount of 25% or more. Offer codes are informational in this prototype and do not change any price.

15.3 Text search

The search box splits the typed text on spaces, commas and full stops, ignores one-letter words, and keeps any product whose name, brand, section name or translated section name contains at least one of the words. Typing resets the section filter and opens the Products page, so searching “logitech” returns the Gaming Mouse and the HD Webcam.

15.4 Voice search and languages

The microphone button uses the browser’s speech-recognition service in the language chosen in the selector and puts the transcript into the search box. Twenty languages are offered, including English, Hindi, Telugu, Tamil, Kannada, Malayalam, Bengali, Marathi, Gujarati, Punjabi, Urdu, Spanish, French, German, Portuguese, Arabic, Chinese, Japanese, Korean and Russian. Interface labels for search, login, offers and section names are translated for English, Hindi, Telugu, Tamil, Spanish and French; other languages fall back to English labels while still driving speech recognition. Voice search works in Chrome and Edge; other browsers show a clear message instead.

16. Authentication and backend API

The application opens on a login screen with two tabs, **Log in** and **Create account**. Registration asks for a full name, email and password; the front end checks that the name has at least two characters, the email looks valid and the password has at least six characters before contacting any server.

16.1 Server mode and demo mode

The API base address is taken from a global setting named SHOPOPT_API and defaults to http://localhost:5000/api. If the server answers, the returned token is stored in memory, the user's name is shown in the header, and the catalogue is loaded from the server. If the server cannot be reached, the screen explains this and offers **Continue in demo mode**, in which nothing is saved. Logging out clears the token, the selection and the result, and returns to the login screen.

16.2 Endpoints used by the front end

Method	Path	Use
POST	/register	Create an account from name, email, password; returns a token and the user.
POST	/login	Sign in; returns a token and the user.
POST	/logout	Ends the session (called when logging out in server mode).
GET	/products	Returns the product list used to replace the built-in catalogue.
POST	/products	Saves a product added through the Add product form.
DELETE	/products/{id}	Removes a deleted product.
POST	/optimize	Records an optimisation run (budget, goal, weights).

Requests carry the header "Authorization: Bearer <token>" after sign-in, and failure responses are expected to contain an "error" text that is shown to the user. If a save to the server fails, the product stays on the page and a note explains that it was saved on this page only.

> **Note on the backend**
>
> The delivered file contains only the front end. The endpoint list above is taken from the calls the front end makes; the server itself is not included, so a developer who wants account storage must implement these seven routes.

17. User guide

17.1 Getting started

1. Open the HTML file in a modern browser. The login screen appears.
2. Log in or create an account. If no server is running, choose **Continue in demo mode**.
3. On the Dashboard, review the budget card, offers and section tiles.

17.2 Browsing and selecting products

1. Open **Products** or tap a section tile or chip.
2. Click a card to select it; selected cards show a green tick and the budget gauge updates.
3. Use **+ Add product** to supply name, price (multiple of 100), rating, utility and discount. Use the camera button to attach your own photo and the bin button to delete a product.

17.3 Finding the best basket

1. Open **Optimizer** and pick a goal. For Combined, drag the sliders to set weights.
2. Type a budget in steps of Rs. 100 and press **Optimize my cart**.
3. Watch the table fill, or use Next step to go one product at a time.
4. Read the Optimization complete card and the selected basket table.

17.4 Exploring the algorithms

1. In **Sorting Lab** choose a key, order and algorithm, then press Sort now.
2. In **Binary Search** enter a price and use Next step to follow each comparison.
3. In **Analytics** press Re-measure, and read the **Complexity** cards for the theory.

17.5 Messages you may see

Message	What to do
Enter a budget above Rs. 0 in steps of Rs. 100.	Use a whole multiple of 100.
Check the values: price must be a positive multiple of 100…	Correct the product details and add again.
Cannot reach the server.	Start the backend or continue in demo mode.

18. Interface, theming and accessibility

18.1 Layout and responsiveness

On wide screens a 230-pixel sidebar holds navigation and the main area is capped at 1,200 pixels. Below 820 pixels wide the sidebar becomes an off-canvas menu opened by a hamburger button, the include and exclude boxes stack vertically, summary figures use two columns, and padding shrinks. The viewport allows content to extend under phone system bars while safe-area insets keep text clear of them. Wide tables scroll sideways inside their own container so the page never scrolls sideways.

18.2 Themes

Colours are defined once as variables. The page follows the system light or dark preference and the button overrides it. Dark mode redefines the same variables, so every card, table and chart adapts without extra code.

18.3 Accessibility features

- Every icon-only button has an accessible name, such as “Open menu”, “Voice search” and “Toggle theme”.
- A visible three-pixel yellow focus outline appears on keyboard focus for buttons and navigation items.
- Interactive controls are at least 38 to 42 pixels high for comfortable touch use.
- Product illustrations are vector graphics with descriptive labels; alerts such as error text use the alert role and the voice status uses the status role.
- Animations and transitions are switched off when the system asks for reduced motion.
- Colour is never the only signal: selected cards show a tick, decisions show words (INCLUDE or EXCLUDE) and the budget gauge reports a percentage in text.

18.4 Visual language

A violet-to-indigo gradient marks primary actions, and each section keeps its own colour across chips, banners, tiles and offer cards: violet for Clothing, blue for Electronics, pink for Beauty, green for Groceries, teal for Medicines and orange for Pets. Typography uses Inter for text and JetBrains Mono for figures. This document uses Bookman Old Style throughout, as requested.

19. Testing and validation

The table lists representative test cases with their expected outcomes. Numerical expectations were obtained by executing the application's own algorithm code on the built-in catalogue.

ID	Test	Expected result	Area
T1	Register with a one-letter name	"Enter your full name."	Validation
T2	Log in with "abc@"	"Enter a valid email address."	Validation
T3	Password of five characters	"Password needs at least 6 characters."	Validation
T4	Optimise utility at Rs. 10,000	Best value 1173, cost Rs. 9,900, 18 products	Knapsack
T5	Optimise combined at Rs. 3,000	29 affordable products, best value 630	Filtering
T6	Budget of 10,050	Budget message (not a multiple of 100)	Validation
T7	Bubble sort of prices	496 comparisons and 353 swaps	Sorting
T8	Merge sort of prices	117 comparisons and 160 writes	Sorting
T9	Search price 2,500	Found at index 27 in 3 steps	Binary search
T10	Search price 2,700	Not found after 5 steps	Binary search
T11	Add product with price 450	Rejected: price must be a multiple of 100	Catalogue
T12	Search "logitech"	Two products: G102 Gaming Mouse, C270 HD Webcam	Search
T13	Open Special offers	Hot deals lists 12 products	Offers
T14	Start with no server running	Server message and demo-mode button	Backend
T15	Resize below 820 pixels	Sidebar becomes a slide-in menu	Responsive

Cross-checks. Because the table is built row by row, the final cell must equal the sum of the selected products' values and the selected prices must total no more than the budget; both properties hold for every run in Table 2. The measured sorting results also agree with the theory: $n(n-1)/2 = 496$ comparisons for bubble and selection sort at $n = 32$.

20. Limitations, future work and conclusion

20.1 Known limitations

- **Price granularity.** The table is built in units of Rs. 100, so prices and budgets must be multiples of 100.
- **One of each.** The 0/1 model allows each product at most once; there is no quantity selection and no rule such as “at least one grocery item”.
- **Section assignment.** Seed products originally labelled Wearables, Accessories, Office and Travel were folded into Electronics (and the Laptop Backpack into Clothing), so the Watch SE and Smart Band appear under Electronics.
- **Offers are display-only.** Codes are not validated and prices are not reduced.
- **Persistence.** In demo mode, added products, deletions and uploaded photos disappear on refresh; the backend is not supplied.
- **Rounding ties.** Integer values can make two baskets equal in score.
- **Partial translation.** Interface text is translated for six languages; the others show English labels.

20.2 Suggested enhancements

- Divide all prices by their greatest common divisor to shrink the table and allow arbitrary prices.
- Add quantity limits (bounded knapsack) and per-section minimums.
- Apply offer codes to prices and include them in the optimisation.
- Persist baskets, products and history on the server and export the final basket.
- Add an automated test suite for the algorithm layer.

20.3 Conclusion

ShopOpt shows how classical algorithms solve an everyday problem. It finds a provably optimal basket within a budget, explains how it did so, and lets the learner compare alternative algorithms on the same data using real counts and timings. Its single-file design, accessible interface and clear separation between the algorithm layer and the presentation layer make it easy to demonstrate, extend and connect to a backend.